#Experiment 5(a): MCE

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification
from sklearn import metrics

d=2
N=100
c=2
#import make_classification dataset with two classes and five features from sklearn
xbar, y = make_classification(n_samples=N, n_features=d, n_redundant=0,
 n_classes=c)
#Normalize input vectors
xbar = xbar/max(np.linalg.norm(xbar, axis=1))

#Output belongs to +1 or -1 rather than +1 and 0 given by dataset
y=2*y-1

# Split the dataset into 80% training and 20% testing sets
xbar_train, xbar_test, y_train, y_test = train_test_split(xbar, y, test_size=0.2)
N_train = np.shape(xbar_train)[0]

# scatter plot for overall dataset, training dataset and testing dataset (consider 2-D features)
plt.scatter(xbar[:, 0], xbar[:, 1], c=y, edgecolors='k', marker='o')
plt.legend([ "Overall_dataset" ])
plt.xlabel('Feature 1')
plt.ylabel('Fetrature 2')
plt.title('DataSet Display (2 Features)')
plt.show()
plt.scatter(xbar_train[:, 0], xbar_train[:, 1], c=y_train, edgecolors='g', marker='*')
```

```
plt.scatter(xbar_test[:, 0], xbar_test[:, 1], c=y_test, edgecolors='r', marker='^')
plt.legend([ "Training_dataset" , "Testing_dataset"])
plt.xlabel('Feature 1')
plt.ylabel('Fetrature 2')
plt.title('DataSet Display (2 Features)')
plt.show()

# Define sign function
def sign_func(x):
   if(x>0):
      return 1
   else:
      return -1

# Sigmoid function
def sigmoid(x):
   return 1 / (1 + np.exp(-x))

product_grad = np.zeros(xbar_train.shape)
# Gradient function
def gradient_func(xbar_train, y_train, weights, N_train):
   for i in range (N_train):
      l = sigmoid(y_train[i]*(np.dot(xbar_train[i],weights)))
      product_grad[i] = y_train[i]*l*(1-l)*xbar_train[i]
   gradient = -product_grad.sum(axis=0)
   return gradient

# Define the logistic training algorithm with gradient descent learning
def logistic_train(x, y, weights, learning_rate, n):
   # number of iterations are limited to 100 for weight update with every dataset point
   for _ in range(100):
```

```
        h = sign_func(np.dot(x, weights))
        if h!=y:
            weights -= learning_rate*gradient_func(xbar_train,y_train,weights,N_train)
            weights = weights/np.linalg.norm(weights)
            n += 1
        else:
            break

    return weights,n

# Initialize the weight vector and normalize it
weights = np.random.rand(xbar_train.shape[1])
weights = weights/np.linalg.norm(weights)

# number of mistakes and learning rate
n=0
learning_rate =0.1

# Train the logistic regression
for i in range (N_train):
    weights,n = logistic_train(xbar_train[i], y_train[i], weights,learning_rate, n)

print("optimized wieght vector for given training dataset")
print(weights)
print("number of mistakes made in training")
print(n)

# test algorithm of test data set
y_pred=np. empty_like(y_test)
for i in range (np.shape(xbar_test)[0]):
    y_pred[i] = sign_func(np.dot(xbar_test[i], weights))
```

```
#plot of testing and predicted dataset
plt.scatter(xbar_test[:, 0], xbar_test[:, 1], c=y_test, marker='o')
plt.scatter(xbar_test[:, 0], xbar_test[:, 1], c=y_pred, marker='^', edgecolors='r')

# Plot decision boundary with testing and predicted datasets
x_min, x_max = plt.xlim()

# Generate points along the decision boundary
x_decision_boundary = np.linspace(x_min, x_max, 100)

# Decision boundary equation for 2 features
y_decision_boundary = -(weights[0] * x_decision_boundary)/ weights[1]
plt.plot(x_decision_boundary, y_decision_boundary, color='blue')
plt.legend([ "Testing_dataset" , "Predicted_dataset", "Decision Boundary"])
plt.xlabel('Feature 1')
plt.ylabel('Fetrature 2')
plt.title('Testing and Predicted DataSet Display (2 Features) with decision boundry')
plt.show()

#Finding misclassification error
misclassifications= y_test-y_pred
no_errors = np.count_nonzero(misclassifications)
per_misclassification = no_errors*100/(np.shape(y_test)[0])
print("no. of errors=")
print(no_errors)
print('% misclassification error =')
print (per_misclassification)

# Print accuracy
```

accuracy = 100-per_misclassification

print(f"Accuracy: {accuracy:.2f}%")

#Find and Plot Confusion Matrix

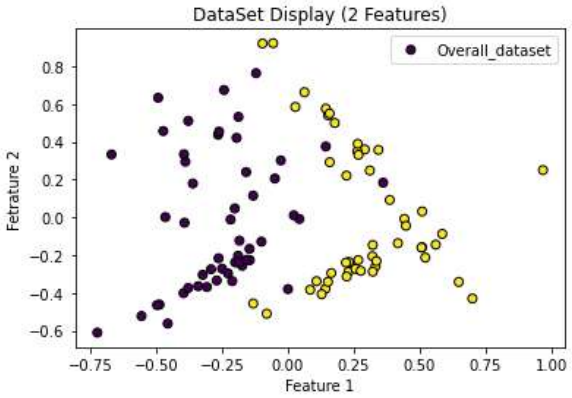
confusion_matrix = metrics.confusion_matrix(y_test, y_pred)

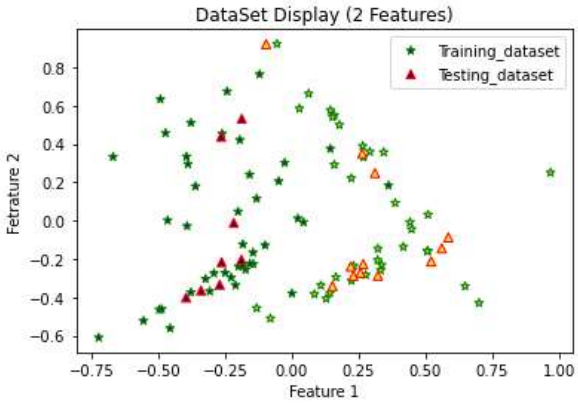
cm_display = metrics.ConfusionMatrixDisplay(confusion_matrix = confusion_matrix, display_labels = [False, True])

cm_display.plot()

plt.show()

#Results:



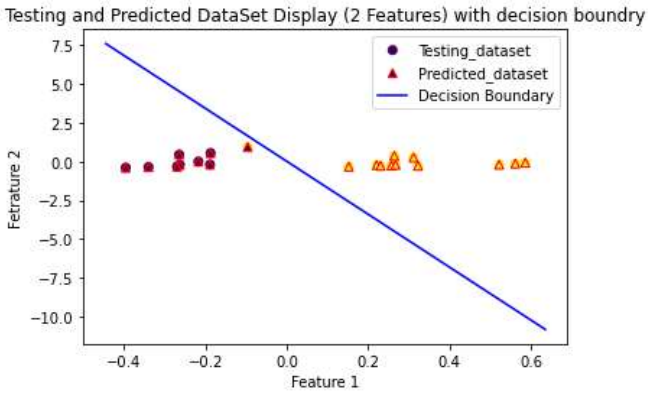


optimized wieght vector for given training dataset

[0.99827774 0.05866478]

number of mistakes made in training

602



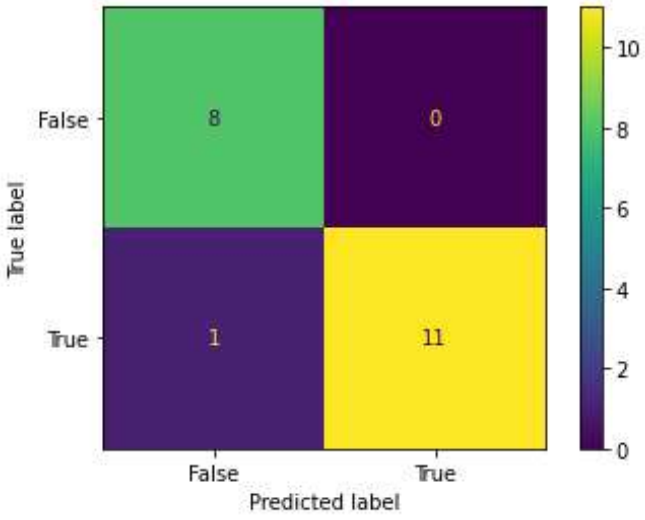
no. of errors=

1

% misclassification error =

5.0

Accuracy: 95.00%



#Experiment 5(b): Logistic Regression

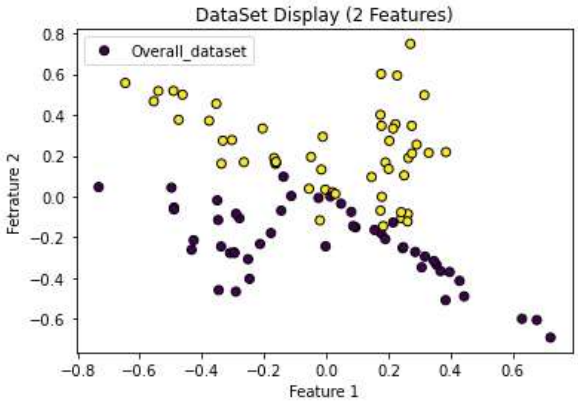
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification
from sklearn import metrics

d=2
N=100
c=2

#import make_classification dataset with two classes and five features from sklearn
xbar, y = make_classification(n_samples=N, n_features=d, n_redundant=0,
 n_classes=c)


#Normalize input vectors
xbar = xbar/max(np.linalg.norm(xbar, axis=1))

#Output belongs to +1 or -1 rather than +1 and 0 given by dataset
y=2*y-1

# Split the dataset into 80% training and 20% testing sets
xbar_train, xbar_test, y_train, y_test = train_test_split(xbar, y, test_size=0.2)
N_train = np.shape(xbar_train)[0]

# scatter plot for overall dataset, training dataset and testing dataset (consider 2-D features)
plt.scatter(xbar[:, 0], xbar[:, 1], c=y, edgecolors='k', marker='o')
plt.legend([ "Overall_dataset" ])
plt.xlabel('Feature 1')
plt.ylabel('Fetrature 2')
```

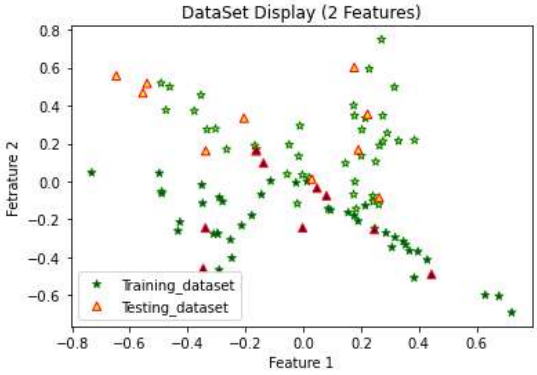
```
plt.title('DataSet Display (2 Features)')
plt.show()
plt.scatter(xbar_train[:, 0], xbar_train[:, 1], c=y_train, edgecolors='g', marker='*')
plt.scatter(xbar_test[:, 0], xbar_test[:, 1], c=y_test, edgecolors='r', marker='^')
plt.legend([ "Training_dataset" , "Testing_dataset"])
plt.xlabel('Feature 1')
plt.ylabel('Fetrature 2')
plt.title('DataSet Display (2 Features)')
plt.show()

# Define sign function
def sign_func(x):
   if(x>0):
      return 1
   else:
      return -1

# Sigmoid function
def sigmoid(x):
   return 1 / (1 + np.exp(-x))



product_grad = np.zeros(xbar_train.shape)
# Gradient function
def gradient_func(xbar_train, y_train, weights, N_train):
   for i in range (N_train):
      l = sigmoid(y_train[i]*(np.dot(xbar_train[i],weights)))
      product_grad[i] = y_train[i]*(1-l)*xbar_train[i]
   gradient = -product_grad.sum(axis=0)
   return gradient
```

```
# Define the logistic training algorithm with gradient descent learning
def logistic_train(x, y, weights, learning_rate, n):
   # number of iterations are limited to 100 for weight update with every dataset point

   for _ in range(100):
      h = sign_func(np.dot(x, weights))
      if h!=y:
        weights -= learning_rate*gradient_func(xbar_train,y_train,weights,N_train)
        weights = weights/np.linalg.norm(weights)
        n += 1
      else:
        break

   return weights,n

# Initialize the weight vector and normalize it
weights = np.random.rand(xbar_train.shape[1])
weights = weights/np.linalg.norm(weights)

# number of mistakes and learning rate
n=0
learning_rate =0.1


# Train the logistic regression
for i in range (N_train):
   weights,n = logistic_train(xbar_train[i], y_train[i], weights,learning_rate, n)

print("optimized wieght vector for given training dataset")
print(weights)
print("number of mistakes made in training")
```

```
print(n)

# test algorithm of test data set
y_pred=np. empty_like(y_test)
for i in range (np.shape(xbar_test)[0]):
    y_pred[i] = sign_func(np.dot(xbar_test[i], weights))



#plot of testing and predicted dataset
plt.scatter(xbar_test[:, 0], xbar_test[:, 1], c=y_test, marker='o')
plt.scatter(xbar_test[:, 0], xbar_test[:, 1], c=y_pred, marker='^', edgecolors='r')

# Plot decision boundary with testing and predicted datasets
x_min, x_max = plt.xlim()

# Generate points along the decision boundary
x_decision_boundary = np.linspace(x_min, x_max, 100)

# Decision boundary equation for 2 features
y_decision_boundary = -(weights[0] * x_decision_boundary)/ weights[1]
plt.plot(x_decision_boundary, y_decision_boundary, color='blue')
plt.legend([ "Testing_dataset" , "Predicted_dataset", "Decision Boundary"])
plt.xlabel('Feature 1')
plt.ylabel('Fetrature 2')
plt.title('Testing and Predicted DataSet Display (2 Features) with decision boundry')
plt.show()

#Finding misclassification error
misclassifications= y_test-y_pred
no_errors = np.count_nonzero(misclassifications)
per_misclassification = no_errors*100/(np.shape(y_test)[0])
```

```
print("no. of errors=")

print(no_errors)

print('% misclassification error =')

print (per_misclassification)


# Print accuracy

accuracy = 100-per_misclassification

print(f"Accuracy: {accuracy:.2f}%")


#Find and Plot Confusion Matrix

confusion_matrix = metrics.confusion_matrix(y_test, y_pred)

cm_display = metrics.ConfusionMatrixDisplay(confusion_matrix = confusion_matrix, display_labels =
[False, True])

cm_display.plot()

plt.show()


# Results:
```



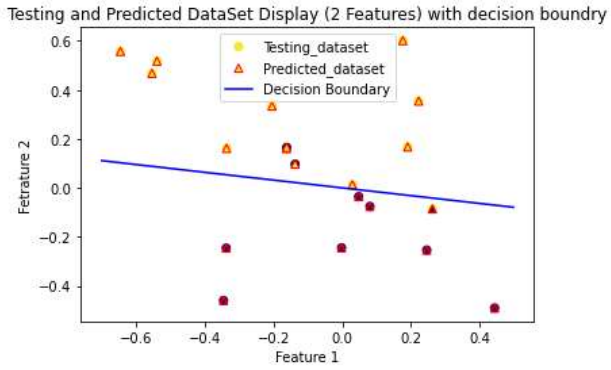


optimized wieght vector for given training dataset

[0.15658069 0.98766517]

number of mistakes made in training

601



no. of errors=

3

% misclassification error =

15.0

Accuracy: 85.00%